

Part 1

Orientation and Project Overview

What LoopHole is, the framework and concepts it is built on, how the repository is organised, how it got into its current state, and which existing documents you can trust.

AUDIENCE

Every team member, especially anyone who has never opened this repository. Read this part first.

COVERS

problem statement, MLIR / Polygeist / Z3 / SymPy primer, glossary, repository tour, project history, documentation trust map, first-day checklist

REPOSITORY STATE

branch main lineage, commit 633c7ad

GENERATED

2026-09-19

SOURCE

docs/handbook/source/01-orientation.md

Contents

Read This First	3
Why this handbook exists	3
Reading paths by role	3
The state of the project on one page	4
The Problem LoopHole Solves	5
Legacy loops and modern accelerators	5
Program lifting	5
Where LoopHole sits in the research landscape	5
The Framework and Concepts You Need	6
MLIR in ten minutes	6
The dialects LoopHole reads and writes	6
Polygeist and cgeist	7
Z3 and SMT solving	7
SymPy	8
Sketches	8
Glossary	9
Repository Tour	10
Top level	10
Inside packages/loophole/	10
Inside docs/	11
How the Project Got Here	12
Timeline	12
How the debt accumulated	12
The Documentation Landscape: What to Trust	14
First-Day Checklist	15

Read This First

Why this handbook exists

LoopHole was built in bursts between February and April 2026 by three people working in parallel "lanes", then restructured into a monorepo in September 2026. Most of the code was produced quickly, often with AI assistance, and each lane documented its own work in its own summary files. The result is a repository that *looks* thoroughly documented (there are more than 60 Markdown files and roughly 350 KB of prose) but where no single document tells a newcomer what the system actually does, where the prose and the code disagree, and where several headline claims ("formally proved", "grade B proof quality", "trusted lane") do not survive contact with the code.

The current team did not write this code. This handbook is the result of reading every source file, every test, the Docker and CI configuration and the main documents, running the full test suite, and running targeted experiments against the real toolchain (Z3, and `mlir-opt` 17 inside the project's Docker image). Every defect called out as "verified" was reproduced, not guessed.

The handbook is split into five parts:

Part	Title	Read it when
1	Orientation and Project Overview (this document)	Day one. Gives you the vocabulary and the map.
2	Architecture and Pipeline	Before changing anything. Explains the data flow, data model, decision logic and configuration.
3	Module Reference	When you work on a specific file. One chapter per module, with internals and drawbacks.
4	Toolchain, Tests, CI and Operations	When you set up, run, test, release or debug.
5	Tech-Debt Audit and Remediation Plan	When you plan work. All verified defects, scored and sequenced, mapped to the Plane tasks.

How the handbook is maintained

The PDFs are generated from Markdown in `docs/handbook/source/` by `docs/handbook/build_handbook.py`. Do not edit the PDFs. Edit the Markdown in the same pull request as the code change that makes a statement outdated, then rebuild. A handbook that drifts from the code becomes one more stale document, which is exactly the debt it is meant to pay down.

Reading paths by role

- **Completely new to compilers and verification:** Part 1 chapters 2 and 3, then Part 2 chapters 2 to 4 (the worked matmul example), then Part 5 chapter 1.
- **Working on the verifier (Lane A, `z3_checker.py`):** Part 2 chapters 5 and 7, Part 3 chapter on `z3_checker.py` and `lifter.py`, Part 5 findings F-01 to F-07 and F-14.
- **Working on parser, emitter or tracer (Lane B):** Part 2 chapters 3 and 8, Part 3 chapters on `affine_extractor.py`, `emitter.py`, `sympy_tracer.py` and `sketch_library.py`, Part 5 findings F-03, F-04, F-05, F-08, F-16.
- **Working on CLI, validation, reports or tests (Lane C):** Part 2 chapters 5, 6 and 9, Part 3 chapters on `cli.py`, `mlir_validator.py`, `replay_checker.py` and the scripts, all of Part 4, Part 5 findings F-09 to F-13.
- **Writing the paper or thesis:** Part 1 chapter 6 and Part 5 chapter 1. Do not quote any "proved" figure from the existing docs until the soundness work in Phase 1 of the remediation plan lands.

The state of the project on one page

What genuinely works today:

- A C or C++ kernel can be compiled to MLIR with Polygeist running inside a reproducible Docker image (`loophole compile`), and that MLIR can be fed to the lifter (`loophole lift`). The one-step `loophole lift-c` combines both.
- For the canonical, correctly written kernels in the fixture set (4x4 matrix multiply, transpose, conv2d, matvec, dot product, elementwise add, row sums and so on) the lifter picks the intended target operation.
- The emitted Linalg MLIR for matmul, transpose, matvec and the conv2d family is accepted by the real `mlir-opt` 17 parser and verifier.
- There is a large test suite (372 tests), batch processing with JSON reports, policy profiles, a counterexample replay tool, and a published Docker image.

What does not work, verified in September 2026:

- **The formal proof is not sound.** The Z3 check rebuilds the source program's arithmetic from a coarse guess ("there is a multiply and an add somewhere") instead of from the actual operations. As a result the tool prints `FORMALLY PROVED - EQUIVALENT` for programs that compute something else: `C = A*B + A` is "proved" equal to elementwise multiply, `C = B - A` is "proved" equal to `A - B`, `c += 2*A[k]*B[k]` is "proved" equal to a plain dot product, and a loop with `step 2` is "proved" equal to the full-tensor operation.
- **Most emitted artifacts are not valid MLIR.** Every `linalg.map` and `linalg.reduce` the emitter produces is rejected by `mlir-opt` 17 with a syntax error, and so are `linalg.dot` and `linalg.conv_1d_ncw_fcw` (rank errors). No StableHLO artifact can be validated at all, because the project's `mlir-opt` has no StableHLO dialect.
- **The "trusted lane" can be satisfied by a fake verifier.** If neither Docker nor `mlir-opt` is available, validation silently falls back to a brace-counting check that accepts nonsense.
- **CI on main is red** since the September restructure, because of a Z3 recursive-function name collision that makes every reduction larger than 32 iterations fail to encode.
- `--no-verify` does not skip verification; it removes Z3's time limit.

Do not trust the historical numbers

The worklog, status and planning documents report figures such as "18 proved / 5 refuted, proof-quality grade B, unexpected refuted 0". Those figures were produced by the unsound checker and count parse failures as "refuted". Treat them as historical records of what the tool printed, not as evidence of correctness.

None of this means the project is a write-off. The architecture is reasonable, the bugs are well localised, and most fixes are small. Part 5 gives the plan, and the Plane board already tracks the most important items as tasks T1 to T11 (LOOPH-9 to LOOPH-19).

The Problem LoopHole Solves

Legacy loops and modern accelerators

A large amount of numerical software (scientific simulation, signal processing, image filters, older machine-learning code) is written as nested scalar loops in C, C++ or Fortran. A matrix multiply looks like this:

```
void matmul(float A[4][4], float B[4][4], float C[4][4]) {
    for (int i = 0; i < 4; ++i)
        for (int j = 0; j < 4; ++j)
            for (int k = 0; k < 4; ++k)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
}
```

A CPU runs this one element at a time. GPUs, TPUs and vector accelerators are fast only when the program is expressed as whole-tensor operations ("multiply these two matrices") that a machine-learning compiler such as XLA or IREE can schedule, fuse and offload. Rewriting legacy code by hand into such operations is slow, error-prone and needs domain expertise.

Program lifting

Lifting is the reverse of compilation: recognising that a low-level loop nest implements a high-level operation and replacing it with that operation. The example above lifts to a single `linalg.matmul` (in MLIR's Linalg dialect) or `stablehlo.dot_general` (in StableHLO, the input format of XLA).

Lifting is only useful if it is **correct**. A lifter that turns $C = B - A$ into $A - B$ silently corrupts results, which is worse than not lifting at all. That is why LoopHole's design has a verification stage: after guessing which operation a loop implements, it tries to *prove* with an SMT solver that the loop and the operation compute the same values. The project calls this "verified lifting".

Where LoopHole sits in the research landscape

The repository's background documents survey the field. The four systems they compare against:

System	Venue	Approach	Reported result
mlirSynth	PACT 2023	Bottom-up enumerative synthesis of MLIR programs	21.6x speedup on TPU
Tenspiler	ECOOP 2024	Verified lifting with SMT via Rosette, sketch-based	105x kernel average
Tensorize	CGO 2025	Symbolic tracing plus algebraic solving	up to 4,102x on GPU
STAGG	PLDI 2025	LLM-guided probabilistic grammar plus A* search	99% accuracy, 3.19 s average

LoopHole is closest to Tenspiler in spirit (a hand-written library of target "sketches", candidate matching, SMT equivalence checking) but takes MLIR as input, produced by Polygeist, and emits MLIR Linalg or StableHLO. It borrows the idea of a symbolic algebra trace from Tensorize, but only uses it to rank candidates, not to prove anything. It does no open-ended synthesis: a loop can only be lifted to one of the 35 operations hard-coded in `sketch_library.py`.

The planning documents describe longer-term research tracks (sparse tensor lifting, Transform-dialect schedule synthesis, dynamic shapes). None of those tracks has any code yet.

The Framework and Concepts You Need

This chapter explains every external technology and every piece of project vocabulary you will meet. If you already know MLIR and SMT solvers, skim the glossary at the end.

MLIR in ten minutes

MLIR (Multi-Level Intermediate Representation) is a compiler framework from the LLVM project. Its key ideas:

- **Operations** are the unit of everything: a function, a loop, an addition and a module are all operations. An operation has a name like `arith.addf`, operands, results, attributes and optionally nested **regions** containing blocks of further operations.
- **SSA values** are written `%name` and are defined exactly once. `%add = arith.addf %c, %mul : f32` defines `%add`.
- **Dialects** group related operations and types under a namespace: `arith.*`, `affine.*`, `linalg.*`. A tool only understands the dialects registered in it, which matters a lot for LoopHole (see the StableHLO validation problem).
- **Textual form**: MLIR can be printed and parsed as text. LoopHole never uses MLIR's C++ or Python APIs for its main work. It reads and writes this text with hand-written string code. That single decision explains a large share of the parser and emitter fragility documented in Parts 3 and 5.
- `mlir-opt` is the standard driver tool. Given a `.mlir` file it parses it, verifies every operation's structural rules and optionally runs transformation passes. LoopHole uses it only as a validator.

The dialects LoopHole reads and writes

Dialect or type	Role in LoopHole	Example
<code>func</code>	Function containers on both input and output side	<code>func.func @matmul(%A: memref<4x4xf32>, ...) { ... return }</code>
<code>affine</code>	Input: loops with affine bounds and array accesses, produced by Polygeist	<code>affine.for %i = 0 to 4 { %a = affine.load %A[%i, %k] : memref<4x4xf32> }</code>
<code>scf</code>	Input: structured control flow (general for-loops), sometimes mixed with affine	<code>scf.for %j = %c0 to %c4 step %c1 { ... }</code>
<code>arith</code>	Input and output: scalar arithmetic and constants	<code>%mul = arith.mulf %a, %b : f32</code>
<code>memref<...> type</code>	Input: Polygeist's pointer-to-array type with shape and element type	<code>memref<?x4xf32> (first dimension dynamic)</code>
<code>linalg</code>	Output target 1: named tensor ops and generic loops over tensors	<code>linalg.matmul ins(%A, %B : ...) outs(%C : ...)</code>
<code>stablehlo</code>	Output target 2: the input format of XLA and IREE	<code>%r = stablehlo.dot_general %A, %B, contracting_dims = [1] x [0]</code>
<code>tensor<...> type</code>	Output: ranked tensor type used by StableHLO emission	<code>tensor<4x4xf32></code>

Two semantic details trip people up. First, Linalg named operations write into an output buffer passed with `outs(...)` and, for accumulating ops like `linalg.matmul`, *add* to its existing contents. StableHLO operations instead return a fresh tensor, so they discard whatever the output buffer held. Second, the Linalg ops emitted by LoopHole use Polygeist's `memref` types instead of `tensor` types. `mlir-opt` 17 parses this, but it is unusual and has not been tested with downstream IREE lowering in this repository.

Polygeist and cgeist

Polygeist is an LLVM-project C/C++ frontend that emits MLIR instead of LLVM IR. Its driver binary is `cgeist` (`cgeist++` for C++). With `-raise-scf-to-affine` it tries to express loops in the `affine` dialect; with `--memref-fullrank` it keeps array parameters as full-rank `memref` values. LoopHole uses a branch called `llvm17bump` that targets LLVM 17. Building it needed several source patches, which the project's Dockerfile applies with Python string replacement (Part 4 walks through each one).

Real Polygeist output differs from the hand-written fixtures in important ways: SSA names are numbers (`%0`, `%arg3`), reductions often use `iter_args` and `affine.yield` instead of `load/add/store`, loop bounds can be `min/max` expressions over affine maps, and a module usually contains several functions. Several of these forms break the parser (Part 5, F-04, F-05 and F-16).

Z3 and SMT solving

Z3 is a Satisfiability Modulo Theories (SMT) solver from Microsoft Research. You give it logical formulas over integers, reals, arrays, uninterpreted functions and so on, and ask whether some assignment makes them all true.

- `sat` means an assignment exists, and Z3 can return it as a **model**.
- `unsat` means no assignment exists.
- `unknown` means Z3 gave up, usually because of a timeout or because the formula uses quantifiers it cannot decide.

Proving by refutation. To prove "P implies Q" you ask Z3 whether "P and not Q" is satisfiable. If it is `unsat`, the implication holds for every input. If `sat`, the model is a counterexample. LoopHole encodes "the loop computes X" as P and "the sketch computes Y" as Q, and checks the implication in both directions. If both are `unsat` it reports **EQUIVALENT**.

Uninterpreted functions stand in for tensors: `A(i, k)` is "whatever value tensor A holds at index (i, k)", with no further assumptions. **ForAll** quantifies over index variables. **RecFunction** defines a recursive function, which LoopHole uses for sums with symbolic or large bounds.

Reals versus floats. LoopHole models every tensor element as a mathematical real number. Real addition is associative and has no overflow, NaN or infinity; IEEE `f32` arithmetic has all of these. A proof over reals therefore says nothing about floating-point corner cases or integer overflow. This is a common, defensible simplification in research prototypes, but it has to be stated whenever the word "proved" is used.

A proof is only as good as its encoding

Z3 proves exactly the formulas it is given. If the code that builds the formula from the source program drops information (operand order, constants, loop steps), Z3 will happily prove a statement about a different program. That is precisely LoopHole's main soundness problem.

SymPy

SymPy is a Python computer-algebra library. LoopHole uses it in three places: to canonicalise index expressions (so `%c2 + (1 + %i)` and `i + 3` compare equal), to infer stride and dilation coefficients for convolutions, and in `sympy_tracer.py` to build a symbolic expression of the loop body and score how well it fits each sketch. The score is only a ranking heuristic; it never decides correctness.

Sketches

A *sketch* (class `OperationSketch` in `sketch_library.py`) is a hand-written description of one target operation, expressed in loop form:

- `dim_names`: the loop index names of the operation's reference loop nest, for example `["m", "n", "k"]` for `matmul`.
- `iterator_types`: for each index, whether it is *parallel* (appears in the output index) or *reduction* (summed or maxed over).
- `indexing_maps`: for each operand, how it is indexed, as strings like `"(m, n, k) -> (m, k)"`.
- `compute_payload`: a coarse label of the inner arithmetic, for example `MULTIPLY_ACCUMULATE (C += A*B)`, `ADD`, `COPY`, `ACCUMULATE_MAX`.
- `num_inputs`, `num_outputs`, `dialect`, `name` and a human-readable `description`.

Lifting means finding the sketch whose reference loop nest is equivalent to the input loop nest, then emitting the named operation.

Glossary

Term	Meaning in this repository
IV (induction variable)	A loop counter such as <code>%i</code> . The parser renames <code>%i</code> to <code>i</code> and <code>%arg3</code> to <code>arg3</code> .
Parallel IV	An IV that appears in the output (store) index. Each value of it writes a different output cell.
Reduction IV	An IV that does not appear in the output index, so its iterations are combined (summed or maxed) into one cell.
Accumulation	The pattern <code>out = out + term</code> , detected by <code>has_accumulation</code> and <code>accumulation_op</code> .
Compute payload	The coarse arithmetic label (<code>ComputePayloadType</code>) used by sketches and, problematically, to rebuild the source side of proofs.
Structural pre-filter	<code>structural_match()</code> : a cheap check that loop count, reduction count and payload are compatible before Z3 runs.
Candidate	A sketch that passed the pre-filter, ranked by SymPy confidence.
Confidence	A 0 to 1 heuristic score from <code>SympyTracer._sketch_confidence</code> . Not a probability and not a proof.
Result state	<code>PROVED</code> , <code>UNPROVED_TIMEOUT</code> or <code>REFUTED</code> (<code>LiftResultState</code>). <code>REFUTED</code> is also used for "no match" and "parse failure".
CheckResult	Z3 outcome per candidate: <code>EQUIVALENT</code> , <code>NOT_EQUIVALENT</code> , <code>TIMEOUT</code> , <code>UNKNOWN</code> , <code>STRUCTURAL_MISMATCH</code> , <code>ENCODE_ERROR</code> .
Strict mode	Accept only <code>PROVED</code> results. Enabled by <code>--strict</code> or the <code>ci-strict</code> profile.
Policy profile	Named defaults for strictness and Z3 timeout: <code>local-explore</code> (alias <code>exploratory</code>) and <code>ci-strict</code> (alias <code>trusted</code>).
Trusted lane	Running with <code>ci-strict/trusted</code> , which additionally forces emitted-artifact validation in <code>batch</code> .
Emitted artifact	The MLIR text produced by the emitter, written as <code>*_lifted.mlir</code> by <code>batch</code> .
Unroll threshold	32. Reductions with at most this many iterations are expanded into explicit sums for Z3.
Guarded unroll	Expanding a reduction with a symbolic bound up to a known maximum, with <code>If(k < N, ...)</code> guards.
Fixture	A test input MLIR kernel, either a string in <code>src/loophole/tests/fixtures.py</code> or a file under <code>tests/fixtures/</code> .
Corpus	<code>tests/fixtures/corpus/</code> : realistic Polygeist-style inputs, several deliberately unsupported.
Counterexample replay	<code>replay_checker.py</code> and <code>loophole replay</code> : re-evaluate index accesses at the values of a Z3 counterexample.
Parametric proof	The A-15 "pilot" that replaces concrete loop bounds with symbols and re-runs the proof.
Canonical StableHLO suite	The set of 11 StableHLO sketch names whose coverage is tracked in reports.
Lane A / B / C	The three work streams (verification, parser and emitter, CLI and workflow) and their file ownership.
A-12, B-15, C-17	Task IDs from the April 2026 execution plans. They appear in test class names, CI step names and docs.
T1 to T11, LOOPH-N	The current (September 2026) Plane tasks in module "Verification Hardening".

Repository Tour

Top level

Path	What it is	Status
<code>packages/loophole/</code>	The only installable Python package: lifter, verifier, emitters, CLI, tests, examples, scripts	Supported, the heart of the project
<code>docker/polygeist/</code>	Dockerfile for the Polygeist + LLVM/MLIR 17 image and <code>toolchain.lock</code>	Supported; the lock file is not actually read by the Dockerfile
<code>.github/workflows/</code>	<code>ci.yml</code> (tests), <code>publish-docker-image.yml</code> (GHCR), <code>export-docker-image-artifact.yml</code> (tarball)	Supported; CI currently failing
<code>docs/</code>	All prose: guides, architecture, background, planning, status, decisions (ADRs), worklog per lane, and this handbook	Mixed trust, see chapter 6
<code>research/</code>	<code>benchmarks/baselines/</code> JSON and Markdown snapshots; empty <code>experiments/</code> and <code>notebooks/</code> templates	Baselines are historical records
<code>papers/</code>	Thesis report and IEEE paper LaTeX sources with committed PDFs	Reflect April 2026 claims
<code>tools/legacy/</code>	Superseded Docker-from-source build and WSL native build scripts	Unsupported, broken paths
<code>.claude/settings.local.json</code>	A committed local AI-assistant permission file	Should not be in version control
<code>README.md</code> , <code>SETUP.md</code> , <code>CONTRIBUTING.md</code> , <code>LICENSE</code>	Entry points	<code>SETUP.md</code> is the most accurate document in the repository

Inside `packages/loophole/`

```

packages/loophole/
  pyproject.toml      package metadata; deps z3-solver, sympy, rich, click (all ">=", unpinned)
  requirements.txt    same deps plus pytest and pytest-cov (duplicated list)
  conftest.py         shared pytest fixtures, MLIR verifier resolution for tests
  src/loophole/
    __init__.py       re-exports the public classes; __version__ = "0.1.0"
    affine_extractor.py Stage 1: MLIR text -> LoopNestInfo (1,301 lines)
    sketch_library.py  35 OperationSketch definitions (752 lines)
    sympy_tracer.py    candidate scoring and convolution recogniser (485 lines)
    z3_checker.py      Stage 3: SMT equivalence checking (1,736 lines)
    emitter.py         Stage 4: Linalg and StableHLO text emitters (1,430 lines)
    lifter.py          orchestrator, result types, policy profiles (690 lines)
    cli.py             Click CLI with 9 commands (1,520 lines)
    polygeist_frontend.py cgeist wrapper, local or Docker (446 lines)
    mlir_validator.py  emitted-artifact validation via mlir-opt (247 lines)
    replay_checker.py   counterexample replay (282 lines)
    parser_corpus_compat.py corpus compatibility classifier (100 lines)
    tests/fixtures.py  39 MLIR kernels as strings (37 in ALL_FIXTURES), used by tests AND `loophole d
emo`
  tests/
    unit/              16 test files
    integration/        11 test files
    fixtures/           .mlir inputs, *_lifted.mlir outputs, corpus/, a12_weak_kernels/, a15_parametri
c/
  examples/           smoke_matmul.c, matmul_inplace.c, matmul_inplace_128.c, run_demo.py
  scripts/            weekly benchmark report, proof-quality summary, corpus analysis, two PowerShel
1 Docker wrappers

```

In total there are about 9,000 lines of library code and about 5,600 lines of tests.

Inside docs/

Folder	Contents
guides/	docker-polygeist.md : image and two-step workflow
architecture/	poc-documentation.md (module walkthrough, written against the February POC), mlir-architecture.md (dialect primer, partly aspirational), architecture.mmd (Mermaid diagram)
background/	problem statement, project overview, state of the art, novel research directions, references
planning/	implementation roadmap, future plan, three-person plans v1 and v2, Docker migration plan
status/	project status (April 16), POC implementation audit (March 31), Docker delivery handoff (April 2)
decisions/	ADR template and ADR 0001 (monorepo layout)
worklog/	lane-a/ (8 records plus a GPU full-workflow log), lane-b/ (17 records), lane-c/ (12 records), development-log.md
handbook/	This handbook: Markdown sources, diagrams, build script, generated PDFs

How the Project Got Here

Timeline

Date	Commits	What happened
2026-02-24 to 02-25	4	Initial commit, background documentation, problem statement, then "Added initial POC code with documentation": 42 files and 8,182 lines in a single commit containing the whole first lifter.
2026-03-04	1	"Add files via upload" (web upload).
2026-03-31	(doc)	POC implementation audit: 96 tests passing; linalg demo 7/10 proved; StableHLO 2/10 proved; demo script broken; refuted results accepted as "partial".
2026-04-02	2	Dockerised Polygeist LLVM 17 pipeline, GHCR publish workflow, image export workflow.
2026-04-03 to 04-04	13	Three-person plan v1, phases 1 to 3. Lane A: trust states and strict mode. Lane B: fallback audit, mixed affine/scf, index normalisation, emitter metadata gates, transpose and conv attributes, artifact validation. Lane C: demo fix, JSON reports, weekly benchmark script, StableHLO sketches, batch UX. One commit added 316,077 lines, mostly a failed MinGW LLVM build directory, later deleted.
2026-04-16	1	Plan v2 (12 weeks, tasks A-11..A-16, B-11..B-16, C-12..C-17), project status and future plan documents.
2026-04-19 to 04-30	13	All v2 tasks closed within two weeks rather than twelve: corpus compatibility, unsupported-form diagnostics, trusted-lane CI, dynamic memref hardening, multi-reduction boundary, counterexample replay, parametric proof pilot, proof-quality grade, IEEE paper and thesis report. Final commit "patches for affine_extractor and other small patches" touched 43 files.
2026-09-13	1	Monorepo restructure (374 files, 315,193 deletions), <code>SETUP.md</code> , ADR 0001. CI turns red.
2026-09-14	(process)	New team adopts Plane + Zulip, lane file ownership, and tasks T1 to T11 from a code review.

Git authors: "Rishab T" (16 commits, lane A work and the initial POC), "Rhithor" (11, lane B), "Yashcomp" (9, lane C), "Schizoid-man" (1, the restructure). The current lane owners are listed in the team's Plane "How we work" page.

How the debt accumulated

Reading the history side by side with the code shows a consistent pattern:

- Task-shaped code instead of design-shaped code.** Each plan task was implemented as an addition to whatever file was nearest, named after the task (`TestDynamicMemrefNormalizationB15`, "A-13: multi-reduction guarded unroll", CI steps named "B-16 guard checks"). Nobody owned the overall shape of `z3_checker.py` or `cli.py`, which grew to 1,700 and 1,500 lines with the same logic repeated three or four times.
- Closure by document.** A task was considered done when a summary Markdown file said so, with a test count. The same fact was then restated in the lane README, the plan's status section, the project status and sometimes the paper. When code changed, none of them were updated, so there are at least eight different "current" test counts in the repository (96, 140, 201, 232, 273, 348, 365, and 364 plus 1 failing).
- Tests that confirm, not challenge.** Every fixture is a correctly written kernel. A checker that approves anything with "a multiply and an add" passes all of them. Several tests assert only `if result.success: ...`, which passes vacuously when the lift fails. The only artifact-validation tests cover the three kernels that happen to produce valid MLIR.

4. **Metrics built on unverified foundations.** The weekly report computes a "proof quality grade" from the prover's own verdicts. Parse failures count as "refuted" and are then excused as "expected corpus refutations". The grade therefore measures the tool's opinion of itself.

5. **Very large commits mixing code, generated artifacts and prose.** This made review impractical and hid regressions: the September red CI is caused by a bug that was already present in April.

The September 2026 process changes (one small PR per task, reviewer from another lane, bug-revealing tests landing first as strict `xfail`, lane file ownership) directly address points 1, 3 and 5. Part 5 proposes additions for points 2 and 4.

The Documentation Landscape: What to Trust

The table rates every major pre-existing document. "Current" means consistent with the September 2026 code; "Historical" means an accurate record of its moment but not of today; "Aspirational" means it describes plans as if they existed.

Document	Trust	Notes
<code>SETUP.md</code>	Current	Accurate commands, env vars and the 364/1 test baseline. The best starting point after this handbook.
<code>README.md</code>	Current	Accurate layout. The one-line pipeline claim "proves equivalence with Z3" needs the caveats in this handbook.
<code>CONTRIBUTING.md</code> , <code>packages/README.md</code> , <code>docs/decisions/*</code>	Current	Describe the post-restructure layout and conventions.
<code>docs/guides/docker-polygeist.md</code>	Mostly current	Says pinned refs "are stored in toolchain.lock" and the Dockerfile "defaults to those pinned refs"; the Dockerfile does not read the lock file and clones a branch head.
<code>docs/architecture/poc-documentation.md</code>	Historical	Friendly walkthrough but written for the February POC: says 26 sketches (now 35) and 10 fixtures (now 39), describes removed behaviour such as high-confidence fallback.
<code>docs/architecture/architecture.mmd</code>	Historical	Same 26-sketch figure; StableHLO emitter shown with 3 ops (now 13 handler entries).
<code>docs/architecture/mlir-architecture.md</code>	Aspirational	Good dialect primer, but describes use of MLIR Python bindings and passes that LoopHole does not use.
<code>docs/background/*</code>	Context	Problem framing and literature. Not about the code.
<code>docs/planning/implementation-roadmap.md</code>	Aspirational	Polybench coverage, symbolic tracer replacing Z3, sparse and transform tracks: none implemented.
<code>docs/planning/three-person-execution-plan-v2-12-weeks.md</code>	Historical	Sections 12.10 to 12.15 are status logs with metrics from the unsound prover.
<code>docs/status/project-status.md</code>	Historical, over-optimistic	April 16 synthesis; its "achievements" list is accurate about what was built, not about whether it is correct.
<code>docs/status/poc-implementation-audit.md</code>	Historical	The most candid older document. Most issues it lists were addressed; its hardcoding table is now obsolete.
<code>docs/status/docker-polygeist-delivery-handoff.md</code>	Historical	Accurate record of the Docker work.
<code>docs/worklog/lane-*/</code>	Historical	Kept as written. Implementation file lists still point at <code>POC_initial_demo/</code> ; lane B README still says the demo script is broken.
<code>docs/worklog/lane-a/fullworkflowtest/README.md</code>	Historical	Real end-to-end C to StableHLO to IREE GPU run (RTX 3060 Ti, 128x128 matmul, about 5.4 ms per iteration). Evidence scripts lived in the git-ignored <code>reports/</code> folder and are not in the repository.
<code>papers/ieee-paper/docs/implementation_findings.md</code>	Historical	Cites <code>packages/loophole/reports/stablehlo_batch_lifting_report_2026-04-06.json</code> , which does not exist in git.
<code>tests/fixtures/corpus/parser_compat_summary.md</code>	Stale	Absolute path from an old machine and an incompatibility class that no longer exists.

Rule of thumb

If a document states a number (tests passing, kernels proved, grades), check its date and assume it is historical. If it states a command, check it against `SETUP.md` or `loophole --help`. If it describes behaviour, check the code, and when they disagree, fix the document in the same PR.

First-Day Checklist

1. Read Part 1 (this document) and skim Part 2 chapters 1 to 4.
2. Build or pull the Docker image and create the virtual environment exactly as in `SETUP.md` sections 2 and 3. On Windows set `PYTHONUTF8=1`.
3. Run the tests from `packages/loophole`: `python -m pytest tests -q`. Expect **372 passed**. Anything else means your environment differs.
4. Run `loophole demo --target linalg` and `loophole lift tests/fixtures/matmul.mlir --target linalg --report`. Compare the output with the worked example in Part 2 chapter 4.
5. Run `loophole compile examples/smoke_matmul.c -o out/smoke.mlir` and open the generated MLIR to see real Polygeist output (numbered SSA names, `iter_args`).
6. Reproduce one soundness bug yourself using Part 5 chapter 2 (for example F-01, `C = B - A`). It takes two minutes and permanently changes how you read "PROVED".
7. Join Plane and Zulip, read the "How we work" page, and pick a task from the current cycle that belongs to your lane.
8. Before your first PR: write the failing test first (strict `xfail` if the fix comes later), keep the PR small, and update any handbook statement your change makes untrue.